

Upwind SCC Graph Ordering for HDG Trace Systems

`dgfem.graph_ordering`

June 26, 2026

Abstract

This note documents the ordering algorithm implemented in `dgfem/graph_ordering.py`. The purpose of the module is to build an experimental permutation for HDG trace unknowns in advection-dominated systems. The ordering is algebraic with respect to the sparse trace system, but it is constructed from mesh-edge connectivity and the sign of the normal advective flux on element faces. The main pipeline is:

1. build a directed graph between mesh-edge blocks using local inflow-to-outflow dependencies;
2. compute strongly connected components (SCCs) with Kosaraju's algorithm;
3. topologically order the SCC condensation graph;
4. lift the ordered edge blocks to a trace degree-of-freedom permutation.

The module also contains sparsity-pattern plotting utilities for comparing the matrix before and after the upwind SCC permutation.

Contents

1	Scope and Design Intent	2
2	Inputs and Core Data Structures	2
2.1	Mesh edge connectivity	2
2.2	Face-normal advective flux	3
2.3	Return containers	3
3	Step 1: Build the Directed Upwind Edge Graph	3
3.1	Graph nodes	3
3.2	Element-local dependencies	4
3.3	Storage	4
4	Step 2: Build CSR Adjacency	4
5	Step 3: Strongly Connected Components	5
5.1	Why SCCs?	5
5.2	Kosaraju algorithm	5
5.3	Correctness property	5
6	Step 4: Build and Order the SCC Condensation DAG	5
6.1	Component graph	5
6.2	Topological order	5

7 Step 5: Lift Component Order to Edge-Block Order	6
8 Step 6: Lift Edge Blocks to Trace DOFs	6
9 Boundary Elimination and Active Edges	6
10 Complexity	7
11 Diagnostics	7
12 Sparse Pattern Plotting	7
13 Interaction with SuperLU ILU	8
14 High-Level Pseudocode	8
15 Function Map	9
16 Practical Usage	9
17 Summary	9

1 Scope and Design Intent

The HDG global unknown is the trace field on mesh edges. For a uniform trace polynomial degree, each global mesh edge carries the same number of trace degrees of freedom, denoted here by

$$n_{\partial} = \text{edg_dof}.$$

The graph-ordering module does not alter element assembly, mesh connectivity, or numerical fluxes. It only computes a permutation of trace unknown indices. The convention matches `dgfem.global_system.solve_global_system`:

$$A_{\pi} = A[\pi, \pi], \quad b_{\pi} = b[\pi],$$

and after solving the permuted system the solution is unpermuted by

$$x[\pi] = x_{\pi}.$$

The ordering is designed to expose approximate upstream-to-downstream structure in advection-dominated HDG trace matrices. It should be viewed as a preconditioner-ordering experiment, not as a guaranteed fill-reducing ordering. In particular, SuperLU's internal orderings such as COLAMD may still be better at high polynomial order because they optimize symbolic fill rather than flow direction.

2 Inputs and Core Data Structures

2.1 Mesh edge connectivity

The primary mesh input is the element-local to global-edge map

$$\text{loc2glob_edge}[K, f] \in \{0, \dots, N_e - 1\},$$

where K is an element index, $f \in \{0, 1, 2\}$ is a local triangle face, and N_e is the number of global mesh edges. In the code this is passed to the Numba kernel as `loc2glob_edge`. For boundary-eliminated trace systems, only non-boundary edge blocks remain active. The helper `_as_active_edges` builds three arrays:

- `active_edge_ids`: sorted global edge ids participating in the reduced system;
- `active_mask`: Boolean mask over all mesh edges;
- `old_to_active`: map from global edge id to active graph node id, with -1 for inactive edges.

2.2 Face-normal advective flux

The second geometric/numerical input is

$$\text{beta_dot_normal}[K, f, q] \approx \beta_h(x_{K,f,q}) \cdot n_{K,f},$$

where q indexes face quadrature points. The graph construction reduces this to one mean face flux per element-local face:

$$\bar{F}_{K,f} = \frac{1}{N_q} \sum_{q=1}^{N_q} \beta_h(x_{K,f,q}) \cdot n_{K,f}.$$

A face is classified as inflow when

$$\bar{F}_{K,f} < -\varepsilon,$$

and as outflow when

$$\bar{F}_{K,f} > \varepsilon,$$

where $\varepsilon = \text{flux_tolerance}$. Faces in the neutral band $[-\varepsilon, \varepsilon]$ are ignored.

2.3 Return containers

The algorithm returns a `GraphOrderingResult`:

<code>edge_order</code>	active global edge ids in graph order,
<code>dof_permutation</code>	trace dof permutation used by the sparse solver,
<code>component_id</code>	SCC id for each active graph node,
<code>component_order</code>	topological order of SCC ids,
<code>diagnostics</code>	graph sizes and timing information.

The diagnostics contain the number of graph nodes, directed graph edges, SCCs, the largest SCC size, the number of cyclic SCCs, the number of nodes in cyclic SCCs, and a timing breakdown.

3 Step 1: Build the Directed Upwind Edge Graph

3.1 Graph nodes

The directed graph nodes are active mesh-edge blocks, not individual trace degrees of freedom. If boundary dofs have not been eliminated, every mesh edge is active. If boundary dofs have been eliminated, only non-boundary edges are active. Therefore the graph vertex set is

$$\mathcal{V} = \{0, \dots, N_a - 1\},$$

where $N_a = \text{active_edge_ids.size}$.

3.2 Element-local dependencies

For each element K , the kernel inspects all local faces. Every active inflow face is connected to every active outflow face in the same element:

$$e_{K,f_{\text{in}}} \longrightarrow e_{K,f_{\text{out}}}.$$

In code, this is done by `_build_upwind_edge_pairs_kernel`. In words:

```
for each element K:
  compute mean face flux F[f] for f=0,1,2
  for each source face f:
    keep it only if active and F[f] < -flux_tolerance
    for each target face g != f:
      keep it only if active and F[g] > flux_tolerance
      append edge active_id(f) -> active_id(g)
```

The direction “inflow to outflow” is the local transport dependency direction: information entering an element through an inflow face contributes to unknowns on outflow faces. The graph can contain duplicate directed edges. Duplicates are accepted because they do not affect reachability or SCC membership, and removing them would add a sorting/unique phase.

3.3 Storage

The kernel emits two arrays

`sources, targets`

with equal length. Directed graph edge m is

$$\text{sources}[m] \rightarrow \text{targets}[m].$$

The maximum number of appended pairs is bounded by $6N_K$, because each triangle has three faces and at most each inflow face connects to the two other faces. Hence the temporary arrays are allocated with length

$$6 \text{ num_elements}.$$

4 Step 2: Build CSR Adjacency

The helper `_build_csr_kernel` converts edge arrays into compressed sparse row adjacency:

$$\text{indptr} \in \mathbb{N}^{N_a+1}, \quad \text{indices} \in \mathbb{N}^{N_{\text{graph edges}}}.$$

The implementation has two passes:

1. count outgoing edges per source node and prefix-sum the counts;
2. fill `indices` using a mutable copy of `indptr`.

The same routine is used for the reverse graph by swapping sources and targets. Duplicate edges remain in the adjacency.

5 Step 3: Strongly Connected Components

5.1 Why SCCs?

If the upwind graph is acyclic, a topological ordering gives an exact upstream to downstream order. If the flow recirculates, some edge blocks are mutually reachable. These cycles cannot be linearly ordered without breaking a dependency. The algorithm therefore groups mutually reachable nodes into SCCs, then orders the SCCs. Inside one SCC, nodes are kept in their original order.

5.2 Kosaraju algorithm

The function `_kosaraju_scc_kernel` implements Kosaraju’s two-pass SCC algorithm using explicit stacks so that it works inside Numba without Python recursion.

The first pass runs depth-first search on the original graph and records nodes by finishing time:

$$\text{finish_order}[0], \dots, \text{finish_order}[N_a - 1].$$

The second pass scans nodes in reverse finishing order and runs DFS on the reverse graph. Each reverse DFS discovers one SCC. The outputs are:

$$\text{component_id}[v] = c, \quad \text{component_sizes}[c] = |\{v : \text{component_id}[v] = c\}|.$$

5.3 Correctness property

For any directed graph, Kosaraju’s algorithm partitions vertices into maximal sets of mutually reachable vertices. Collapsing each SCC into one component produces a directed acyclic graph. That acyclicity is what makes the next topological-order step well-defined.

6 Step 4: Build and Order the SCC Condensation DAG

6.1 Component graph

The function `_component_edges_kernel` maps each original graph edge

$$u \rightarrow v$$

to an SCC edge

$$C(u) \rightarrow C(v).$$

Self-edges, where $C(u) = C(v)$, are discarded. The result is the condensation graph. It is a DAG by construction.

6.2 Topological order

The function `_topological_order_kernel` computes indegrees in the condensation DAG and applies Kahn’s queue algorithm:

```
queue = all components with indegree zero
while queue is not empty:
    pop a component c
    append c to the order
    for each outgoing edge c -> d:
```

```

decrement indegree[d]
if indegree[d] becomes zero:
    push d

```

The code checks that the number of ordered components equals the number of components. Failure would indicate a bug, because the condensation graph of an SCC partition must be acyclic.

7 Step 5: Lift Component Order to Edge-Block Order

The function `_node_order_from_components_kernel` produces an ordered list of graph nodes. It reserves contiguous ranges for components according to the topological component order, then inserts original node ids into the range of their component. Thus:

- components are topologically ordered;
- nodes inside one SCC keep original active-node order.

The active-node order is then converted back to global mesh-edge ids:

```
edge_order = active_edge_ids[node_order].
```

8 Step 6: Lift Edge Blocks to Trace DOFs

The sparse trace matrix is indexed by individual trace degrees of freedom. If edge block e has n_∂ local trace dofs, its trace dof range is

$$en_\partial, en_\partial + 1, \dots, en_\partial + n_\partial - 1.$$

The helper `_trace_dof_permutation_kernel` receives ordered active-node positions, not global edge ids. For each ordered active node position a , it appends the contiguous local block

$$an_\partial, an_\partial + 1, \dots, an_\partial + n_\partial - 1.$$

This is exactly the correct indexing for reduced systems where active edge blocks have been compacted into consecutive reduced-system coordinates.

For a full, non-reduced trace system, active nodes are all edges in natural global edge order. Then the active-node position and global edge id coincide. For a boundary-eliminated system, `active_edges` must be passed so the active edge positions match the reduced-system matrix.

9 Boundary Elimination and Active Edges

The ordering routine is used in two common modes:

Full penalty system. Boundary trace dofs remain in the system. The active edge set is all mesh edges:

```
active_edges = None.
```

Boundary-eliminated system. Prescribed boundary trace dofs are removed before the solve. The reduced matrix contains only non-boundary edge blocks. In this case the caller builds

$$\text{active_edges} = \{e : e \notin \partial\Omega\}.$$

The graph is built using global edge ids, but all graph nodes are compacted through `old_to_active`. The resulting dof permutation is therefore in reduced-system coordinates and can be applied directly to the reduced matrix.

10 Complexity

Let N_K be the number of elements, N_a the number of active edge blocks, and M_g the number of directed graph edges after local inflow to outflow construction. For triangular elements,

$$M_g \leq 6N_K.$$

Ignoring duplicate graph edges, the algorithmic costs are:

graph pair construction	$O(N_K N_q),$
CSR construction	$O(N_a + M_g),$
Kosaraju SCC	$O(N_a + M_g),$
component DAG construction	$O(M_g),$
topological order	$O(N_{\text{comp}} + M_{\text{comp}}),$
dof lift	$O(N_a n_{\partial}).$

The graph-ordering cost is usually small compared with HDG matrix assembly and incomplete factorization. It is also almost independent of the polynomial degree p , except for the final dof lift through $n_{\partial} = p + 1$.

11 Diagnostics

`GraphOrderingDiagnostics` reports:

<code>num_nodes</code>	$N_a,$
<code>num_directed_edges</code>	$M_g,$
<code>num_components</code>	$N_{\text{comp}},$
<code>largest_component_size</code>	$\max_c C_c ,$
<code>cyclic_components</code>	$\#\{c : C_c > 1\},$
<code>cyclic_nodes</code>	$\sum_{c: C_c > 1} C_c .$

For a nearly acyclic upwind graph, one typically sees

$$\text{largest_component_size} = 1, \quad \text{cyclic_nodes} = 0.$$

For recirculating flows, SCCs larger than one indicate trace-edge blocks whose flow dependencies are mutually coupled.

The timing object `GraphOrderingTimings` records the wall-clock time for graph pair construction, CSR construction, SCC, DAG construction, topological ordering, dof permutation, and total ordering.

12 Sparse Pattern Plotting

The module also provides two plotting helpers:

<code>save_sparse_pattern_plot</code>	plot one sparse matrix pattern,
<code>save_upwind_reordered_matrix_patterns</code>	plot before and after $A[\pi, \pi]$.

The marker area is chosen by `sparse_pattern_marker_area`. The intended law is inverse in the number of nonzeros:

$$s = s_{\text{ref}} \frac{N_{\text{ref}}}{\text{nnz}(A)}.$$

The current implementation clips this value and applies an additional practical floor, so very large matrices do not disappear completely in PNG output. Very large matrices are also stride-sampled to `max_plot_points`; the marker area is still computed from the original $\text{nnz}(A)$, not from the sampled count.

13 Interaction with SuperLU ILU

The upwind SCC permutation is not a symbolic fill-reducing ordering. It is a physics-motivated block ordering. If the sparse solver is SuperLU’s `spilu`, then its `permc_spec` option matters:

- **NATURAL** forces SuperLU to respect the supplied matrix ordering. This can expose the benefit of a good upwind order, but it can also be fragile at high polynomial degree.
- **COLAMD** lets SuperLU apply a fill-reducing column ordering. This may override much of the upwind-ordering effect, but it can be far more robust when local trace blocks become large.

Thus the SCC ordering should be benchmarked together with the ILU column permutation strategy. A small edge-block graph can look excellent while the lifted high-order dof matrix still has poor symbolic fill under **NATURAL**.

14 High-Level Pseudocode

```
function upwind_scc_trace_ordering(mesh, beta_dot_normal, edg_dof,
                                active_edges=None, flux_tolerance=0):
    active_edge_ids, active_mask, old_to_active = active_edges_to_maps(...)

    sources, targets =
        build_upwind_edge_pairs(mesh.loc2glob_edge,
                                beta_dot_normal,
                                active_mask,
                                old_to_active,
                                flux_tolerance)

    node_order, component_id, component_order, component_sizes, timings =
        strongly_connected_component_order(num_active_edges, sources, targets)

    edge_order = active_edge_ids[node_order]
    dof_permutation = lift_edge_order_to_trace_dofs(node_order, edg_dof)

    return GraphOrderingResult(edge_order,
                                dof_permutation,
                                component_id,
                                component_order,
                                diagnostics)
```

15 Function Map

Function	Role
<code>_build_upwind_edge_pairs_kernel</code>	Build directed inflow-to-outflow edge graph.
<code>_build_csr_kernel</code>	Convert directed edge arrays to CSR adjacency.
<code>_kosaraju_scc_kernel</code>	Compute SCCs with iterative Kosaraju DFS.
<code>_component_edges_kernel</code>	Build condensation graph edges between SCCs.
<code>_topological_order_kernel</code>	Topologically order the condensation DAG.
<code>_node_order_from_components_kernel</code>	Convert component order to node order.
<code>_trace_dof_permutation_kernel</code>	Lift ordered edge blocks to trace dofs.
<code>strongly_connected_component_order</code>	Public graph-only SCC ordering routine.
<code>upwind_scc_trace_ordering</code>	Public HDG trace-edge ordering routine.
<code>sparse_pattern_marker_area</code>	Compute nnz-dependent marker area for plots.
<code>save_sparse_pattern_plot</code>	Save one matrix sparsity pattern figure.
<code>save_upwind_reordered_matrix_patterns</code>	Save before/after upwind permutation figures.

16 Practical Usage

From the advection-reaction CLI, the current diagnostic path can generate the matrix patterns without entering the expensive global solve:

```
python -m dgfem.adv_rea -p 7 --lc 0.03 \  
--boundary-mode eliminate \  
--matrix-pattern-only \  
--matrix-pattern-max-points 2000000
```

This writes figures such as:

```
dgfem/matrix_patterns/*_before_upwind.png  
dgfem/matrix_patterns/*_after_upwind.png
```

17 Summary

The ordering in `graph_ordering.py` is a block-level flow ordering: construct local inflow-to-outflow dependencies, compress cycles with SCCs, topologically order the acyclic component graph, and lift the result to trace dofs. The graph construction is inexpensive and mostly independent of polynomial order. Its effect on ILU depends on how the block ordering interacts with the symbolic fill behavior of the sparse factorization.